For this assignment I worked alone. For part a (AVL tree implementation), I began to strategize by analyzing the code file to understand what the code was doing and formulate edge cases for insertion, deletion, and tree balancing that I thought of as going through the code. Then, I wrote my first .avl test files that portrayed the simple tests such as rotations with a three node tree, left and right insertions, deletions of nodes, and large tree insertion and deletion. I kept running code coverage with new tests to see which ones would give me higher coverage. For part b (libpng), I used existing example PNG files and also added additional images from the library's contrib folder. Using these images for a starting base, I also experimented with variations in color depth and palette types to increase statement coverage by continuously checking results with gcov. For part c (JFreeChart), I modified starter Java tests to cover API calls and creating test classes that considered multiple chart types and datasets by using AI to help me create a list of test scenarios to consider.

This assignment was more difficult than I thought it would be, especially considering the differences with white-box and black-box testing that were seen in the different parts of the project. Part a was most like what I have done where I used direct inspection of the small Python codebase and reasoned about edge cases and needed to cover all of the important branch statements and scenarios. Part b was more difficult with the large C library. Coverage gains were difficult to achieve at first and I had to use a lot of external assistance to gain insight on how I could tweak images to increase coverage, rather than reading the code line by line as in part a. Part c required careful API exploration and I had to think about input combinations to maximize coverage without having graphical output, but it was more straightforward than part b. A common theme across the parts was trial and error and consulting external sources as I gained more coverage and it became harder to increase coverage. Part b was most different because I needed to look at the problem from an image perspective, while the other two parts were code-based tests that I was more familiar with. Overall I learned that while black-box testing is much harder to implement for me, I can understand why it may be more beneficial to do it because being able to gain coverage without knowing the implementation means that the program functions exactly how the developers intended it to, which is the goal of software.

Links used:

- http://www.libpng.org/pub/png/libpng.html
- https://stackoverflow.com/questions/tagged/libpng
- https://stackoverflow.com/questions/tagged/gcov
- https://coverage.readthedocs.io/
- https://www.jfree.org/jfreechart/api/javadoc/
- https://en.wikipedia.org/wiki/Portable_Network_Graphics
- https://en.wikipedia.org/wiki/AVL_tree
- https://chat.openai.com